CSC311: Introduction to Machine Learning

Project Report

Submitted by

Group 46965

Abhinn Kaushik, Elvis Chen, Charles Chen

University of Toronto
Winter 2026

1 Executive Summary

We explored four model families—Logistic Regression, Random Forest, Support Vector Machine (SVM), and a multimodal LSTM neural network—to classify paintings from student survey responses. **Logistic Regression** (L2, $C = 1$) achieved the best performance with a test accuracy of **89.02%** and macro F1 of 0.892, followed closely by SVM (88.24%) and LSTM (88.43%), with Random Forest trailing at 87.45%. Logistic Regression outperformed the alternatives because the dataset's 622-dimensional feature space—dominated by sparse TF-IDF text features combined with strongly discriminative Likert ratings—is naturally suited to a well-regularized linear model. Across five random data splits, Logistic Regression averaged 90.0% test accuracy with the lowest variance ($\sigma = 1.0\%$), confirming its robustness. We estimate our best model's accuracy on the unseen test set to be **89%**.

2 Data Exploration

Dataset Overview

The training set contains 1,686 rows collected from 562 students, each of whom answered 14 survey questions for 3 paintings, with exactly 562 samples for each painting (33.3%).

The 14 features fall into four distinct types:

- **Numerical** (3 features): emotion intensity (1–10 scale), number of prominent colours, and number of objects noticed. Emotion intensity is roughly symmetric (mean 6.3, std 2.2), while colours (median 3, max 100) and objects (median 3, max 100) are right-skewed with outlier values.
- **Likert scale** (4 features): agreement ratings (1–5) for feeling sombre, content, calm, and uneasy. These are stored as strings (e.g. "4 - Agree") and must be parsed to integers.
- **Multi-select categorical** (3 features): room placement, viewing companion, and season association. Responses are comma-separated lists of categories (e.g. "Bedroom,Living room").
- **Free-text** (4 features): emotional description, willingness to pay (in dollars), food analogy, and soundtrack description. The dollar field is especially noisy, containing currency symbols, words, and non-numeric entries.

Data Quality Issues

Missing values. Every feature column has approximately 4–5% missing values. We identified 68 rows where nearly all features are missing and one entry that is entirely Chinese. These rows carry almost no information and are effectively noise.

Outliers. In the colours and objects columns, 24 and 17 values exceed three times the interquartile range. The dollar column is the noisiest, as 90.8% of non-null entries can

be parsed as numbers, values range from \$0 to over \$$10^{15}$, making the raw distribution unusable without transformation.

Inconsistencies. The dollar column contains mixed formats ("\$5", "300 dollars.", "5 000 000\$", "a"), requiring careful parsing and filtering. The multi-select columns use comma separation without consistent spacing.

Preprocessing

We apply the following transformations, all fitted exclusively on the training set:

1. **Likert parsing:** extract the leading integer from Likert each response string (e.g. "4 - Agree" $\rightarrow$ 4). Missing values are imputed with the training-set mode.
2. **Dollar parsing:** strip currency symbols, unit words, and whitespace, then convert to float. Unparseable entries become missing and are imputed with the training-set median to prevent losing out on too many data points. We clip extreme values at the 1st and 99th percentiles, then apply a $\log(1 + x)$ transform to reduce right skew.
3. **Outlier clipping:** colours and objects are clipped at the 1st and 99th percentiles of the training set.
4. **Standardization:** all eight numeric and Likert features are standardized to zero mean and unit variance. This is important for logistic regression, where feature scales affect the regularization penalty.
5. **Multi-hot encoding:** each categorical variable's options (5 rooms, 5 companion types, 4 seasons = 14 binary features) are expanded into its own indicator variable.
6. **TF-IDF vectorization:** the three free-text columns (description, food, and soundtrack) are each converted to TF-IDF vectors with sublinear term-frequency scaling, English stop-word removal, a minimum document frequency of 2, and a cap of 200 features per column (600 total) using `scikit-learn`. This captures discriminative vocabulary while keeping dimensionality manageable.

Data Splitting and Leakage Prevention

To prevent splitting one student's response across groups and causing data leakage, we split by `unique_id` (student) rather than by row, using grouped shuffle splits. The resulting split is 70% training (392 students, 1,176 rows), 15% validation (85 students, 255 rows), and 15% test (85 students, 255 rows). All three splits retain perfect class balance (33.3% per painting). The test set is reserved and not used during exploration or model development.

Key Insights

The Likert features show the strongest per-class separation (Figure 1), and season association was also highly discriminative (Figure 3).

The correlation matrix (Figure 2) reveals strong pairwise correlations among the Likert features: content and calm are positively correlated ($r = 0.69$), while both are negatively

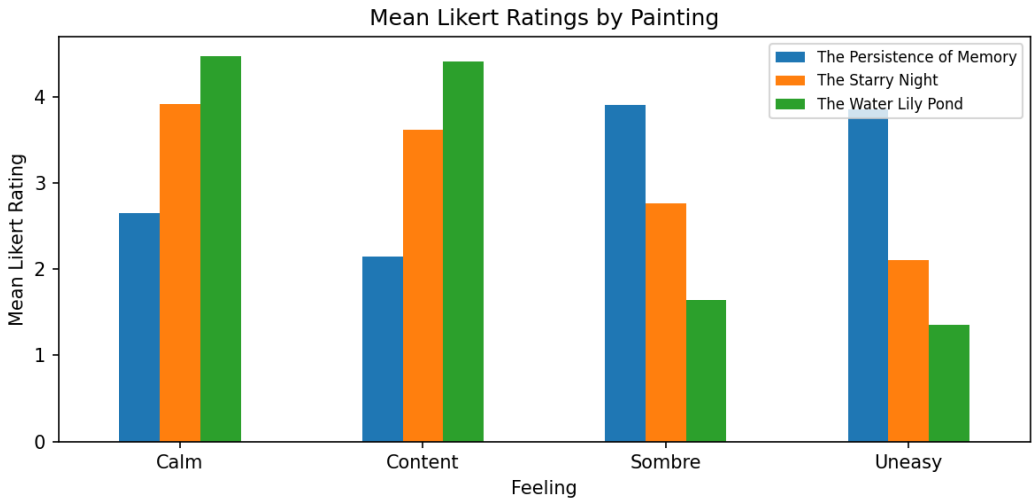

Figure 1: Mean Likert ratings by painting. The four emotional dimensions separate the three classes: *Water Lily Pond* scores highest on calm and content, while *Persistence of Memory* scores highest on sombre and uneasy.

correlated with uneasy ($r = -0.72$ and -0.71). This suggests some redundancy, though each feature still contributes unique signal.

These patterns suggest that even a linear model should achieve reasonable accuracy from the structured features alone, while the free-text features may provide additional gains by capturing painting-specific vocabulary.

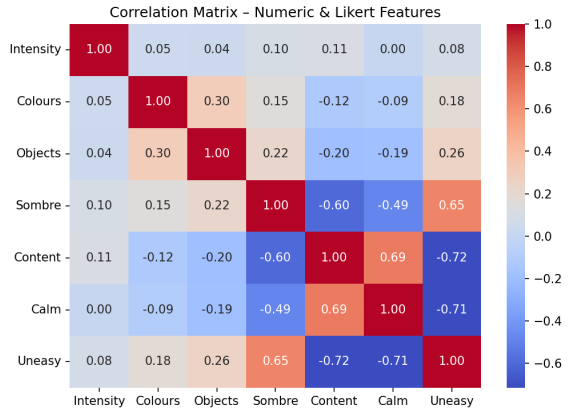

Figure 2: Correlation matrix of numeric and Likert features.

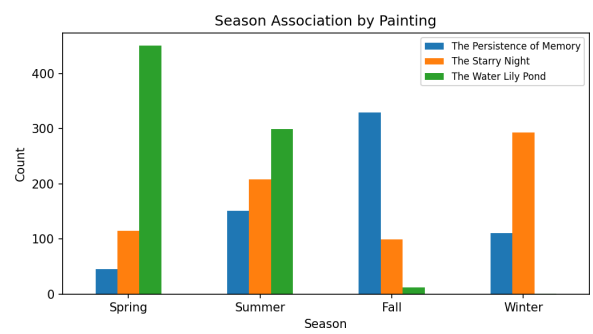

Figure 3: Season association counts by painting.

3 Methodology

Model Families

We explored the following four model families, chosen to cover a range of complexity and inductive biases:

1. **Logistic Regression (multinomial).** A linear model that operates directly on the full feature vector (standardized numeric/Likert features, one-hot categoricals, and TF-IDF text features). It is well-suited to this dataset because (i) the high-dimensional, sparse TF-IDF features naturally favour linear classifiers, (ii) the Likert and season features already show approximately linear separation between classes (see Figure 1), and (iii) L2 or L1 regularization helps prevent overfitting in the presence of 600+ features. Additionally, the learned coefficients are directly interpretable, allowing us to identify which words or features are most predictive.
2. **Decision Tree (with bagging/random forest extension).** A model that recursively partitions the feature space using axis-aligned splits. We start with a single decision tree and extend it to a Random Forest to reduce variance. This family is suitable for our dataset because
   (a) Trees naturally handle mixed feature types without requiring standardization or normalization.
   (b) Recursive splits can capture non-linear interactions between features that a linear model misses.
   (c) Random Forest extension aggregates many decorrelated trees, which will reduce overfitting.
   (d) Built-in feature importance scores allow us to identify which features contribute most to classification.
3. **Neural Network (multimodal LSTM).** A multimodal neural network that combines sequential text modeling via a Long Short-Term Memory (LSTM) layer with structured tabular inputs. Unlike the logistic regression and decision tree approaches, which rely on bag-of-words (TF-IDF) text representations that discard word order, the LSTM processes token sequences directly, allowing it to capture sequential and contextual patterns in the free-text responses. The structured branch handles numeric, Likert, and categorical features through a separate fully connected layer, and the two branches are fused before the final classification head. This architecture is well-suited to our dataset because it exploits both the rich text data and the discriminative structured features simultaneously.
4. **Support Vector Machine.** An alternative linear model which also operates on the full feature vector as described in 3.1. It is well suited to the dataset as (i) The SVM performs well in high dimensional spaces, where the ratio of number of features (after TF-IDF encoding) to sample size is high. (ii) The SVM benefits from the ability to use the kernel trick to represent non-linear relationships. (iii) it is inherently more tolerant to outliers due to its loss function; this is notable as the numerical sample data contains a wide range of values, particularly in the dollars feature. However, it loses its interpretability, as it does not output the probability of each classification.

Preprocessing per Model

The preprocessing pipeline differs by model family:

- **Logistic Regression:** standardized numeric/Likert features, log-transformed and standardized dollar values, multi-hot encoded categoricals, TF-IDF text features (as described in Data Exploration).
- **Decision Tree / Random Forest:** We applied the same preprocessing as logistic regression, but skipped standardization as decision trees do not depend on feature scale. Text features were excluded to simplify the pipeline, as structured features alone already achieved strong performance.
- **Neural Network (LSTM):** structured features are preprocessed identically to logistic regression (standardization + multi-hot). For text, the three free-text fields (description, food, soundtrack) are concatenated with a `[SEP]` token, lowercased, stripped of punctuation, and tokenized by whitespace. A vocabulary is built from the training set only (tokens with frequency ≥ 2, max size 5,000). Sequences are padded or truncated to a fixed length of 64 tokens, with special `<PAD>` and `<UNK>` tokens.
- **Support Vector Machine:** uses the same preprocessing as Logistic Regression, as both are linear models sensitive to scaling and require encoded text.

Optimization

- **Logistic Regression:** we used `sklearn`'s L-BFGS solver for the multinomial case, with L2 regularization controlled by the inverse regularization strength C. No learning rate schedule is needed since L-BFGS is a quasi-Newton method.
- **Decision Tree / Random Forest:** For the single decision tree, we used `sklearn`'s `DecisionTreeClassifier`. For the random forest, we used `RandomForestClassifier`. No learning rate schedule is needed as tree-based models do not use gradient-based optimization.
- **Neural Network (LSTM):** we used the Adam optimizer with learning rate 5×10^{-4}, weight decay 10^{-4}, and batch size 32 for up to 100 epochs. Early stopping monitored validation loss with patience 10 to prevent overfitting. Dropout ($p = 0.2$) was applied after the LSTM and before the output layer. The model architecture consists of two branches: a *text branch* (embedding layer with dimension 100 $\rightarrow$ 1-layer LSTM with hidden size 128 $\rightarrow$ final hidden state) and a *structured branch* (fully connected layer with 64 units, ReLU). The two representations are concatenated (128 + 64 = 192 dimensions), passed through a fusion layer (FC 128, ReLU, dropout), and a softmax output layer (3 classes). Cross-entropy loss was used throughout.
- **Support Vector Machine:** we used `sklearn`'s linear and RBF kernels with a One-vs-One (OvO) strategy. We used squared L2 penalty for regularization, controlled by inverse regularization strength C, and γ to control sensitivity to noise/outliers for the RBF kernel.

Validation and Hyperparameter Tuning

We used the grouped 70/15/15 train/validation/test split described in Data Exploration. All hyperparameter tuning was performed by evaluating on the *validation set only*; the test set remained untouched until final evaluation. We tuned all four model families before selecting the best, to ensure a fair comparison.

The hyperparameters tuned for each model are shown in Table 1. We performed grid search over these values, selecting the configuration that maximized validation accuracy. We also verified that results were stable across five random seeds.

Model	**Hyperparameter**	**Search range**	**Best**
Logistic Regression	Penalty type	$\{\text{L1}, \text{L2}\}$	L2
	Regularization C	$\{0.01, 0.1, 1, 10, 100\}$	1
Random Forest	Number of trees	$\{100, 200, 300\}$	300
	Max depth	$\{10, 20, \text{None}\}$	None
	Max features per split	$\{0.3, 0.5, \sqrt{p}\}$	0.5
	Min samples per leaf	$\{1, 2, 5\}$	1
LSTM	LSTM hidden size	$\{64, 128\}$	128
	Embedding dimension	$\{50, 100\}$	100
	Dropout rate p	$\{0.2, 0.3\}$	0.2
	Learning rate	$\{10^{-3},\ 5 \times 10^{-4}\}$	5×10^{-4}
SVM	Kernel type	$\{\text{linear}, \text{rbf}\}$	linear
	Regularization C	$\{0.1, 1, 10, 100\}$	0.1
	Kernel coefficient γ	$\{\text{scale}, \text{auto}, 0.1, 0.01\}$	N/A

Table 1: Hyperparameters, search ranges, and best values for each model family.

Evaluation Metrics

We used the following metrics to evaluate and compare all four models:

- **Accuracy:** the primary metric, since the classes are perfectly balanced and correct classification is the end goal.
- **Macro-averaged F1 score:** the harmonic mean of precision and recall, averaged equally across all three classes. This metric is more informative than accuracy alone because it penalizes models that perform well on some classes but poorly on others. Since we have balanced classes, macro F1 and accuracy will be similar in magnitude, but F1 highlights per-class weaknesses.
- **Per-class precision and recall:** reported in a classification report to identify which paintings are most often confused with each other.
- **Confusion matrix:** to visualize the specific misclassification patterns (e.g. whether *The Starry Night* is more often confused with *Persistence of Memory* or *Water Lily Pond*).

Implementation Details

All models were implemented using `scikit-learn` (logistic regression, random forest, SVM) and standard Python libraries (`pandas`, `numpy`). Text vectorization for the linear and tree models used `sklearn.feature_extraction.text.TfidfVectorizer`. The LSTM-based neural network was implemented in `PyTorch`, with custom `Dataset` and `DataLoader` classes that handle padded token sequences and structured features jointly. A shared preprocessing module (`preprocess.py`) ensured identical feature pipelines across Logistic Regression, SVM, and Random Forest. The final prediction script (`pred.py`) loads saved model parameters and uses only `numpy` and `pandas` as required by the project constraints.

4 Results

Model Performance

Table 2 summarizes the performance of all four model families on the primary split (seed = 42). Logistic Regression achieved the highest test accuracy (89.02%) and macro F1 (0.892). SVM and LSTM performed comparably at ∼88%, while Random Forest trailed at 87.45%, likely due to the exclusion of text features from its pipeline. We selected **Logistic Regression** as the final model due to its highest accuracy, lowest variance across seeds, and simplicity of deployment.

Model	Train Acc.	Val Acc.	Test Acc.	Test Macro F1
Logistic Regression (L2, $C = 1$)	95.34%	88.49%	**89.02%**	**0.892**
SVM (linear, $C = 0.1$, OvO)	92.11%	87.70%	88.24%	0.884
LSTM (hidden=128, emb=100)	—	88.41%	88.43%	∼0.88
Random Forest (300 trees)	98.56%	83.73%	87.45%	0.874

Table 2: Performance comparison across model families. Best values in bold.

Error Analysis

The confusion matrices for the two best models (Table 3) reveal a consistent error pattern: misclassification *into The Starry Night*. In Logistic Regression, 23 of 28 total test errors involve other classes being predicted as *The Starry Night* (11 from *Persistence of Memory*, 12 from *Water Lily Pond*); the SVM shows an almost identical pattern (20 of 30 errors). This gives *The Starry Night* the highest recall (0.94 for LR) but the lowest precision (0.78), suggesting it occupies a middle ground in the feature space—evoking both the unease of *Persistence of Memory* and the calm of *Water Lily Pond*. Direct confusion between *Persistence of Memory* and *Water Lily Pond* is negligible (0–1 errors each direction), confirming these two classes are well-separated.

	Logistic Regression			**SVM**		
	Pers.	Starry	Lily	Pers.	Starry	Lily
Persistence of Memory	**74**	11	0	**74**	10	1
The Starry Night	3	**80**	2	5	**77**	3
Water Lily Pond	0	12	**73**	1	10	**74**

Table 3: Test-set confusion matrices (rows = true, columns = predicted).

Test Performance Estimate

We estimate our best model's accuracy on the unseen test set to be:

$$\boxed{89\%}$$

To justify this estimate, we re-ran the full pipeline across five random seeds (42, 123, 7, 99, 2024). Logistic Regression achieved a mean test accuracy of **90.0%** with a standard deviation of only **1.0%**—the lowest variance among all models (SVM: $89.3 \pm 0.9\%$, RF: $87.4 \pm 1.9\%$). Validation and test accuracies were consistently aligned within 1–3 percentage points across all seeds, confirming the model does not overfit to any particular split. We round down from 90.0% to 89% as a conservative adjustment, since the actual test set is compiled from TAs and instructors whose responses may differ in style from the student training data.

5 Contributions and Learning

- **Abhinn Kaushik:** I was responsible for designing, implementing, and evaluating the multimodal LSTM model. This included building the text processing pipeline, integrating structured features, and tuning key hyperparameters such as hidden size, embedding dimension, dropout, and learning rate. The most important lesson I learned was that while neural networks can model complex patterns like sequential text, they do not always outperform simpler models when the dataset is relatively small and well-structured.
- **Elvis Chen:** I was responsible for training and evaluating the decision tree model. I tuned its hyperparameters and compared its performance with the other models. The most important lesson I learned was that model performance depends mostly on careful tuning, and simpler, interpretable models can still work well.
- **Charles Chen:** I was responsible for compiling our results and the final report, as well as the full pipeline for the SVM model.

6 References

[1] C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.

[2] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*, 2nd ed., Springer, 2009.

[3] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.